Week 4: Customer Order Management and Order History System

Aim of the Project

Definition: The aim is to create and manage customer orders, maintain accurate order details, establish table relationships, apply data-integrity constraints, insert records, and generate customer and product order reports.

Create Orders and Order_Details Tables

Definition: This step creates the two tables required for storing order-level information and the individual products included in each order.

Orders Table

Definition: The Orders table stores one record for each customer order, including details such as Order ID, Customer ID, Order Date, Total Amount, and Order Status.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT PK_ORDERS PRIMARY KEY(Order_ID);

Table altered.
```

```
SQL> DESC ORDERS;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ORDER_ID                                  NOT NULL VARCHAR2(7)
 CUSTOMER_ID                                        VARCHAR2(7)
 ORDER_DATE                                         DATE
 TOTAL_AMOUNT                                       NUMBER(10,2)
 ORDER_STATUS                                       VARCHAR2(20)
```

```
SQL> DESC ORDER_DETAILS;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ORDER_DETAIL_ID                           NOT NULL VARCHAR2(7)
 ORDER_ID                                           VARCHAR2(7)
 PRODUCT_ID                                         VARCHAR2(7)
 QUANTITY                                           NUMBER
```

Order_Details Table

Definition: The Order_Details table stores the products belonging to each order, including Order Detail ID, Order ID, Product ID, and Quantity.

```
SQL> SELECT O.Order_ID,O.Order_Date,O.Total_Amount,OD.Product_ID,OD.Quantity FROM ORDERS O JOIN ORDER_DETAILS OD ON O.Order_ID=OD.Order_ID;

ORDER_I ORDER_DAT TOTAL_AMOUNT PRODUCT   QUANTITY
------- -------- ------------ ------- ----------
ORD0001 24-SEP-26        25000 P100001          2
ORD0001 24-SEP-26        25000 P100002          1
ORD0002 24-SEP-26        15000 P200002          2
ORD0002 24-SEP-26        15000 P100001          1
ORD0002 24-SEP-26        15000 P100002          2
```

```
SQL> SELECT * FROM ORDERS;

ORDER_I CUSTOME ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
------- ------- --------- ------------ --------------------
ORD0001 CUS0001 24-SEP-26        25000 Pending
ORD0002 CUS0002 24-SEP-26        15000 Shipped
```

Establishing Relationships

Definition: Relationships connect related tables using keys. Order_Details is connected to Orders through Order_ID and to Products through Product_ID using foreign keys.

```
SQL> CREATE TABLE ORDER_DETAILS(Order_Detail_ID VARCHAR2(7),Order_ID VARCHAR2(7),Product_ID VARCHAR2(7),Quantity NUMBER);

Table created.
```

```
SQL> INSERT INTO ORDER_DETAILS VALUES('OD00001','ORD0001','P100001',2);

1 row created.

SQL> INSERT INTO ORDER_DETAILS VALUES('OD00002','ORD0001','P100002',1);

1 row created.

SQL> INSERT INTO ORDER_DETAILS VALUES('OD00003','ORD0002','P200002',2);

1 row created.

SQL> INSERT INTO ORDER_DETAILS VALUES('OD00004','ORD0002','P100001',1);

1 row created.

SQL> INSERT INTO ORDER_DETAILS VALUES('OD00005','ORD0002','P100002',2);

1 row created.
```

Applying Constraints for Data Integrity

Definition: Constraints are rules that ensure valid and consistent data. Primary Key, Foreign Key, NOT NULL, UNIQUE, and CHECK constraints prevent invalid or duplicate data where required.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_ORDER_DETAILS_PRODUCT FOREIGN KEY(Product_ID) REFERENCES PRODUCT(Product_ID);

Table altered.
```

```
SQL> SELECT SUM(Total_Amount) AS Grand_Total FROM ORDERS;

GRAND_TOTAL
-----------
      40000
```

```
SQL> SELECT o.Customer_ID,o.Order_ID,o.Order_Date,o.Total_Amount,o.Order_Status,od.Product_ID,od.Quantity FROM ORDERS o JOIN ORDER_DETAILS od ON o.Order_ID=
od.Order_ID;

CUSTOME ORDER_I ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         PRODUCT   QUANTITY
------- ------- --------- ------------ -------------------- ------- ----------
CUS0001 ORD0001 24-SEP-26        25000 Pending              P100001          2
CUS0001 ORD0001 24-SEP-26        25000 Pending              P100002          1
CUS0002 ORD0002 24-SEP-26        15000 Shipped              P200002          2
CUS0002 ORD0002 24-SEP-26        15000 Shipped              P100001          1
CUS0002 ORD0002 24-SEP-26        15000 Shipped              P100002          2
```

Inserting Values

Definition: INSERT statements are used to add order and order-detail records into the database tables.

```
SQL> SELECT COUNT(*) FROM ORDERS;

  COUNT(*)
----------
         2
```

```
SQL> SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE FROM USER_CONSTRAINTS WHERE TABLE_NAME IN ('ORDERS','ORDER_DETAILS');

CONSTRAINT_NAME                C
------------------------------ -
PK_ORDERS                      P
PK_ORDER_DETAILS               P
FK_ORDER_DETAILS_ORDER         R
FK_ORDER_DETAILS_PRODUCT       R
```

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_ORDER_DETAILS_ORDER FOREIGN KEY(Order_ID) REFERENCES ORDERS(Order_ID);

Table altered.
```

After Inserting Values

Definition: After inserting records, SELECT and aggregate queries are used to verify the data and obtain useful order information.

```
SQL> INSERT INTO ORDERS VALUES('ORD0001','CUS0001',TO_DATE('24-09-2026','DD-MM-YYYY'),25000,'Pending');

1 row created.

SQL> INSERT INTO ORDERS VALUES('ORD0002','CUS0002',TO_DATE('24-09-2026','DD-MM-YYYY'),15000,'Shipped');

1 row created.
```

```
SQL> SELECT OD.Order_ID,P.Product_Name,OD.Quantity FROM ORDER_DETAILS OD JOIN PRODUCT P ON OD.Product_ID=P.Product_ID;

ORDER_I
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY
----------
ORD0001
Laptop
         2

ORD0001
Mobile Phone
         1

ORDER_I
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY
----------

ORD0002
Keyboard
         2

ORD0002
Laptop

ORDER_I
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY
----------
         1

ORD0002
Mobile Phone
```

```
SQL> SELECT Order_Status,COUNT(*) AS Total_Orders FROM ORDERS GROUP BY Order_Status;

ORDER_STATUS         TOTAL_ORDERS
-------------------- ------------
Shipped                         1
Pending                         1
```

```
SQL> SELECT * FROM ORDERS WHERE Total_Amount=(SELECT MIN(Total_Amount) FROM ORDERS);

ORDER_I CUSTOME ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
------- ------- --------- ------------ --------------------
ORD0002 CUS0002 24-SEP-26        15000 Shipped
```

Report 1: Customer Order History

Definition: This report displays a customer's order history with Customer Name, Order ID, Order Date, Total Amount, and Order Status.

```
SQL> SELECT Order_ID,SUM(Quantity) AS Total_Quantity FROM ORDER_DETAILS GROUP BY Order_ID;

ORDER_I TOTAL_QUANTITY
------- --------------
ORD0001              3
ORD0002              5
```

```
SQL> SELECT o.Customer_ID,NULL AS Customer_Name,o.Order_ID,o.Order_Date,od.Product_ID,od.Quantity,p.PRIC AS Unit_Price,(od.Quantity*p.PRIC) AS Subtotal FROM
 ORDERS o JOIN ORDER_DETAILS od ON o.Order_ID=od.Order_ID JOIN PRODUCT p ON od.Product_ID=p.Product_ID ORDER BY o.Customer_ID,o.Order_Date;

CUSTOME C ORDER_I ORDER_DAT PRODUCT   QUANTITY UNIT_PRICE   SUBTOTAL
------- - ------- --------- ------- -------- ---------- ---------
CUS0001   ORD0001 24-SEP-26 P100002         1      25000     25000
CUS0001   ORD0001 24-SEP-26 P100001         2      50000    100000
CUS0002   ORD0002 24-SEP-26 P100001         1      50000     50000
CUS0002   ORD0002 24-SEP-26 P100002         2      25000     50000
CUS0002   ORD0002 24-SEP-26 P200002         2       3000      6000
```

Report 2: Productwise Order Report

Definition: This report summarizes product ordering activity, such as the number of times each product was ordered and the total quantity sold.

```
SQL> SELECT O.Order_ID,O.Customer_ID,OD.Product_ID,OD.Quantity FROM ORDERS O JOIN ORDER_DETAILS OD ON O.Order_ID=OD.Order_ID;

ORDER_I CUSTOME PRODUCT   QUANTITY
------- ------- -------- ----------
ORD0001 CUS0001 P100001          2
ORD0001 CUS0001 P100002          1
ORD0002 CUS0002 P200002          2
ORD0002 CUS0002 P100001          1
ORD0002 CUS0002 P100002          2
```

```
SQL> SELECT Product_ID,COUNT(*) AS Times_Ordered FROM ORDER_DETAILS GROUP BY Product_ID;

PRODUCT TIMES_ORDERED
------- -------------
P100001             2
P200002             1
P100002             2
```

Report 3: Customer Purchase Analysis

Definition: This report analyzes customer purchasing activity using order count, total spending, and average order value.

Customers with Maximum Orders

Definition: This analysis counts the orders placed by each customer and identifies the customer or customers with the highest number of orders.

```
SQL> SELECT Customer_ID,COUNT(*) AS Total_Orders FROM ORDERS GROUP BY Customer_ID;

CUSTOME TOTAL_ORDERS
------- ------------
CUS0002            1
CUS0001            1
```

```
SQL> SELECT Product_ID,MAX(Quantity) AS Max_Quantity FROM ORDER_DETAILS GROUP BY Product_ID;

PRODUCT MAX_QUANTITY
------- ------------
P100001            2
P200002            2
P100002            2
```

Total Spending by Each Customer

Definition: This analysis calculates the total amount spent by each customer by adding the Total_Amount values of their orders.

```
SQL> SELECT Product_ID,SUM(Quantity) AS Total_Quantity FROM ORDER_DETAILS GROUP BY Product_ID;

PRODUCT TOTAL_QUANTITY
------- --------------
P100001              3
P200002              2
P100002              3
```

```
SQL> SELECT Customer_ID,COUNT(*) AS Total_Orders FROM ORDERS GROUP BY Customer_ID;

CUSTOME TOTAL_ORDERS
------- ------------
CUS0002            1
CUS0001            1
```

Average Order Value

Definition: Average Order Value is the average amount spent per order. It is calculated from the order amounts and the number of orders.

```
SQL> SELECT COUNT(*) FROM ORDER_DETAILS;

  COUNT(*)
----------
         5
```

```
SQL> SELECT Order_Status, SUM(Total_Amount) AS Total_Amount FROM ORDERS GROUP BY Order_Status;

ORDER_STATUS         TOTAL_AMOUNT
-------------------- ------------
Shipped                     15000
Pending                     25000
```